분야 AI/SW 기초 구분 웹 기초와 프론트엔드 학습시간 80시간 필수/선택 선택

버튼 누르면 화면이 스르륵 바뀌는 요즘 웹사이트 만들기

문제기술

기술적 설명



1. 미션 소개

어떤 상태가 어디에 있어야 하는지, 언제 리렌더링이 일어나는지 이론만으로는 감이 잘 안 잡힙니다. 상태 관리가 왜 어려운지는 SPA를 직접 만들어봐야 알 수 있습니다. 컴포넌트 구조와 데이터 흐름을 결과물로 확인하는 것, 그게 이번 미션의 목표입니다.

React는 전 세계에서 가장 널리 사용되는 프론트엔드 UI 라이브러리입니다. 컴포넌트 기반으로 화면을 구성하고, 상태(state)와 이벤트를 중심으로 UI를 선언적으로 업데이트하는 방식이 핵심입니다.

이 미션에서는 React로 하나의 SPA 서비스를 직접 구현하며, 컴포넌트 설계, 라우팅, 상태 관리, 비동기 데이터 처리의 흐름을 체계적으로 경험합니다. 단순히 화면을 그리는 것이 아니라, "사용자 이벤트 → 상태 변화 → 렌더링 변화"가 연결되는 React의 동작 원리를 결과물로 확인합니다.

또한 Supabase 또는 Firebase를 최소 범위로 연동하여, 실제 서비스에서 자주 등장하는 로딩/에러/빈 상태를 일관된 UI 패턴으로 처리하는 경험을 쌓습니다. 이 미션의 평가는 백엔드 구현이 아니라 React 관점의 구조와 데이터 흐름 완성도를 중심으로 진행됩니다.

2. 최종 결과물

다음 조건을 만족하는 React 기반 웹 서비스 1개를 완성한다. (백엔드는 Supabase 또는 Firebase 중 하나를 선택)

1. 라우팅이 있는 SPA

- 최소 5개 이상의 페이지 라우트가 존재한다.

(예: / , /login , /items , /items/:id , /items/new , /profile)

2. 핵심 데이터 CRUD

- “서비스의 핵심 데이터(게시물/할 일/리뷰/예약 등)”를 등록/조회/수정/삭제할 수 있다.
- 목록 페이지와 상세 페이지가 존재한다.

3. React 폼 기반 입력 경험

- 등록/수정 화면에 폼이 있고, 유효성 검증 + 에러 표시 + 제출 중 상태가 UI로 드러난다.

4. 상태 관리가 설계된 UI

- 로딩/에러/빈 상태가 모든 핵심 화면에서 일관된 방식으로 처리된다.
- 컴포넌트 단위로 상태가 적절히 분리되어 있다.

5. 배포

- 외부에서 접속 가능한 제출 URL(예: Vercel/Netlify 등)로 배포되어야 한다.
- 소스 코드 공유 링크(GitHub 레포지토리 URL)를 함께 제출한다.
- README.md에 실행 방법(로컬 설치/실행 명령어)과 사용한 기술 스택을 명시한다.

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- React에서 컴포넌트가 왜 필요한지, 그리고 본인이 어떤 기준으로 컴포넌트를 쪼갰는지 설명할 수 있다.
- props 와 state 의 차이, 그리고 상태를 어디에 두었는지(상향/하향 흐름) 설명할 수 있다.
- useEffect 가 언제 실행되고, 어떤 의존성으로 동작하며, 데이터 요청과 어떤 관계가 있는지 설명할 수 있다.
- 비동기 흐름에서 로딩/성공/실패/빈 상태를 React UI로 어떻게 표현했는지 설명할 수 있다.
- “하나의 기능”을 만들기 위해 라우팅 → 컴포넌트 → 상태 → 이벤트 → 렌더링이 어떻게 연결되는지 설명할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 프로젝트 기본 구성

- React 프로젝트로 시작한다.
- 폴더 구조가 최소한 다음 역할을 분리한다.
    - pages (라우트 단위 화면)
    - components (재사용 컴포넌트)
    - hooks 또는 lib (커스텀 훅/유틸)
- 공통 레이아웃(헤더/네비게이션)이 주요 페이지에 적용되어야 한다.
- “서비스 주제(데이터 종류)”는 학습자가 직접 정하되, 단일 핵심 데이터 CRUD가 가능한 수준으로 제한한다.

2. 라우팅(React Router 등)

- 최소 5개 라우트가 동작해야 한다.
- 목록/상세 라우트가 포함되어야 한다. (예: `/items`, `/items/:id`)
- 잘못된 주소 접근 시 Not Found 페이지가 있어야 한다.
- 네비게이션을 통해 주요 라우트로 이동 가능한 링크가 제공되어야 한다.

3. 컴포넌트 설계

- 최소 8개 이상의 “재사용 컴포넌트”가 존재해야 한다.
    - 예: `Button`, `Input`, `Card`, `EmptyState`, `ErrorState`, `Loading`, `ItemList`, `ItemForm` 등
    - 재사용 컴포넌트는 최소 1개 이상의 prop을 받아 동작이나 표시가 달라지는 컴포넌트를 의미한다.
- “페이지 컴포넌트”와 “UI 컴포넌트”가 섞이지 않도록 분리되어야 한다.
- 동일한 UI 패턴(로딩/에러/빈 상태)은 페이지마다 따로 만들지 않고 재사용 가능한 컴포넌트로 통일되어야 한다.

4. 상태 관리(React 중심)

- 아래 상태를 React 방식으로 관리해야 한다.
    - 폼 입력 상태 (controlled input)
    - 목록/상세 데이터 상태
    - 로딩/에러 상태
- 데이터 조회/갱신 흐름 중 최소 1개 이상은 커스텀 훅으로 분리되어야 한다.
    - 예: `useItems()`, `useItemDetail(id)` 등

5. CRUD

- CRUD는 반드시 Supabase 또는 Firebase의 원격 데이터 기준으로 동작해야 한다.
    - 목록 조회: 리스트 UI가 렌더링된다.
    - 상세 조회: 라우트 파라미터로 특정 데이터를 불러와 렌더링한다.
    - 등록/수정: 폼 입력 → 제출 → 성공 시 이동/갱신 흐름이 존재한다.
    - 삭제: 삭제 후 목록 갱신 또는 이동 흐름이 존재한다.

6. 폼 UX

- 등록/수정 폼에 필수값 검증이 존재해야 한다. (예: 제목/내용 비어있으면 제출 불가)
- 에러 메시지가 입력 필드 근처 또는 상단에 표시된다.
- 제출 중에는 버튼 비활성화 또는 스피너 등 “진행 중” 상태가 표시된다.
- 요청 실패 시(네트워크/권한/기타 오류) 사용자에게 실패 사실이 화면에 표시되어야 한다.

7. React 이벤트 & 렌더링

- 사용자 이벤트(클릭/입력/제출 등)가 상태 변경으로 이어지고, 렌더링이 변하는 흐름이 명확해야 한다.
- 상태 변경이 렌더링 변화로 이어지는 지점이 최소 3군데 이상 명확해야 한다.
    - 예: 필터 변경 → 목록 변경, 입력값 변경 → 미리보기 변경, 저장 성공 → 알림 표시 등

8. 배포

- 배포된 URL에서 아래 흐름이 모두 동작해야 한다.
    - 목록/상세 조회
    - 등록/수정/삭제
- 환경변수 등 설정 누락으로 인해 배포 환경에서 기능이 일부라도 동작하지 않으면 요구사항을 충족하지 못한다.

5. 보너스 과제 (선택)

1. 전역 상태 도입

- 로그인 사용자, 테마, 알림 중 하나를 전역 상태(Context 등)로 관리해본다.

2. 성능 최적화

- 불필요한 리렌더링을 줄이기 위해 메모이제이션 (useMemo/useCallback/React.memo) 중 1개 이상을 적용해본다.

3. 인증 추가

- Supabase/Firebase Auth로 로그인 흐름을 붙이고, 보호 라우트를 적용해본다.

개발환경

6. 개발 환경

- React 18 이상

제약조건

7. 제약 사항

- 프레임워크: React 기반으로 구현한다.
- 핵심 목표: React의 컴포넌트/상태/이벤트/비동기 렌더링 학습이 중심이다.
- 백엔드
  - Supabase 또는 Firebase 사용한다.
  - 백엔드 고급 기능(권한/RLS/Rules, 복잡한 관계 설계)은 필수가 아니다.
- 구조
  - 라우팅이 존재해야 한다.
  - 페이지/컴포넌트/훅(또는 lib)이 분리되어야 한다.
- 기능 범위
  - UI 고퀄리티보다 “React 구조와 데이터 흐름”이 우선이다.
  - 백엔드 서버를 직접 구현하는 방식은 요구하지 않는다.
- 언어 및 스타일링
  - TypeScript 사용은 선택 사항이다. JavaScript만으로 구현해도 무방하며, TypeScript를 사용할 경우 가산점은 없으나 코드 품질 향상에 도움이 될 수 있다.

- 스타일링 방식은 자유롭게 선택한다. 아래 중 어떤 방식을 사용해도 된다.
  - 순수 CSS / CSS Modules
  - Tailwind CSS
  - styled-components / Emotion 등 CSS-in-JS
  - UI 라이브러리 (MUI, Chakra UI, Ant Design 등)
- 반응형 디자인은 필수가 아닌 선택 사항이다.

- 환경변수 및 보안
  - API Key 등 민감 정보는 .env 파일에 저장하고, .gitignore 에 .env 가 포함되어 있는지 반드시 확인한다.
  - API Key가 포함된 코드를 GitHub에 절대 푸시하지 않는다.
  - 배포 시에는 Vercel/Netlify 등의 대시보드에서 Environment Variables를 별도로 등록한다.

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구, 디자인은 얼마든지 달라도 된다.

- /items 에서 카드 리스트가 보이고, 로딩 중에는 스피너가 보인다.
- 리스트 항목 클릭 시 /items/123 로 이동하고 상세가 뜬다.
- /items/new 에서 폼을 작성하고 저장하면 목록 또는 상세로 이동한다.
- 빈 데이터면 “표시할 데이터가 없습니다.”가 보인다.
- 에러면 “요청에 실패했습니다. 다시 시도하세요.”가 보인다.